



B.TECH II YEAR I SEM --- BACKEND WEB TECHNOLOGIES

QUESTION BANK

MODULE-I

Short Questions

1. What is meant by Backend development?
2. Explain briefly about Node.js.
3. What is the importance of Express.js?
4. What is the purpose of handling HTTP requests in web applications?
5. Explain briefly about Routing.
6. What is meant by Server?
7. What is Server Architecture?
8. What is Error Handling in Express.js? Why is it important?
9. What is NPM? Mention any two uses of NPM in Node.js
10. What is the difference between CommonJS and ES Modules in Node.js?

Long Questions

1. Explain Backend Development. Discuss its role in web application architecture?
2. Explain CommonJS and ES Modules in detail.
3. Explain Server Architecture in detail and discuss its major features.
4. Explain NPM Package Management in detail. Discuss its features and commands.
5. Explain briefly about Request and Response Objects in Express.js.
6. Briefly explain about Error Handling in Express.js.
7. Explain the steps involved in setting up an HTTP server using Node.js and Write a suitable program to create an HTTP server.
8. Explain how HTTP requests are handled in Node.js. Discuss different HTTP methods with examples.
9. Explain the concept of routing in Express.js with different routes.
10. Discuss Express.js in detail with its features, advantages and example program.



MODULE-II

Short Questions

1. Define SQLite. Mention any two features of SQLite.
2. What is a Database Schema?
3. Expand CRUD and list its operations.
4. What is a RESTful API?
5. Differentiate between Query Parameters and Route Parameters.
6. What is the purpose of a Primary Key in a database?
7. What is request validation? Why is it important in API development?
8. What is response formatting in REST APIs? Mention two commonly used response formats.
9. What is API testing? Name any two tools used for testing REST APIs.
10. Define query optimization?

LONG QUESTIONS:

1. Explain SQLite integration in Node.js with suitable code examples?
2. Describe CRUD operations in Node.js using SQLite with examples?
3. Explain Query Writing and Query Optimization techniques in Node.js?
4. Explain RESTful API Development in Node.js using Express.js with an example?
5. Discuss Query Handling in Express.js using query parameters?
6. Explain the REST Principles in detail with suitable examples?
7. Explain how to implement searching, filtering, sorting, and pagination in a RESTful API using Node.js, Express.js, and SQLite?
8. Explain request validation in REST APIs. Demonstrate how to validate user input before inserting data into a SQLite database?
9. Explain common debugging techniques for Node.js, Express.js, and SQLite applications?
10. Explain Query Parameters and Route Parameters in Express.js. Develop suitable API examples demonstrating both types of parameters?



MODULE-III

Short Questions

1. What is JWT (JSON Web Token) authentication?
2. What are the three parts of a JWT?
3. What is the purpose of the JWT signature?
4. Define authentication and authorization.
5. List any two authentication strategies used in web applications.
6. What is middleware?
7. Mention any two uses of middleware functions in web applications.
8. What is custom middleware?
9. What is error logging, and why is it important?
10. Differentiate between authentication and authorization with one example each.

LONG QUESTIONS:

1. Explain JWT (JSON Web Token) authentication in detail. Describe the structure of a JWT, token generation, verification, expiration, and its advantages and limitations.
2. Discuss various authentication strategies used in web applications. Compare session-based authentication and JWT authentication.
3. Explain the complete user authentication process in a web application, from user registration and password storage to login.
4. What are middleware functions? Explain their purpose, working mechanism, execution flow, and importance in web application development with suitable examples.
5. Explain the implementation of authentication middleware for protecting routes. Describe how middleware validates JWT tokens and allows or denies access to protected resources.
6. What is Custom Middleware Development? Explain how to create and implement custom middleware for authentication, request validation, logging, and access control.
7. Explain Error Logging in web applications. Discuss different types of errors, logging levels, error-handling middleware.



8. Design an authentication and authorization architecture for a web application using JWT, middleware, user management, and role-based authorization. Explain the flow with a suitable diagram.
9. Explain error handling and security middleware integration in a web application. Describe how authentication errors, authorization errors, validation errors, and server errors should be handled and logged.
10. Develop a complete authentication and authorization workflow using JWT and middleware. Explain user registration, login, JWT generation, protected routes, custom middleware, and error handling.

MODULE-IV

Short Questions

1. What is fs module in Node.js?
2. What is the difference between fs.readFile() and fs.createReadStream()?
3. What is a Readable Stream?
4. What is a Writable Stream?
5. What is a Duplex Stream?
6. What is a Transform Stream?
7. What is the purpose of the pipe() method?
8. What is a Buffer in Node.js?
9. How is the "error" event used in stream error handling?
10. What is the difference between Promise-based and Callback-based asynchronous file operations?

LONG QUESTIONS:

1. Explain the Node.js File System (fs) module and discuss the various file operations such as creating, reading, appending, renaming, and deleting files with suitable examples.
2. Explain the directory operations available in Node.js using the fs module. Discuss how to create, read, and delete directories with appropriate methods and examples.
3. What are streams in Node.js? Explain their advantages and discuss the different types of streams—Readable, Writable, Duplex, and Transform Streams.



4. Explain how to work with Readable and Writable file streams in Node.js. Describe the data, end, and finish events with suitable examples.
5. Explain Duplex Streams in Node.js. How do their readable and writable sides work independently? Illustrate with an example.
6. What are Data Transformation Streams? Explain how a Transform Stream modifies data while it is being transferred. Develop an example to convert file content to uppercase.
7. Explain error handling in Node.js streams. Discuss the use of `.on("error")`, `pipeline()`, with suitable examples.
8. Explain Stream Piping in Node.js. Discuss how data is transferred from a Readable Stream to a Writable Stream and explain its advantages with examples.
9. Explain Buffer Management in Node.js. Discuss Buffer creation, reading, writing, converting Buffers to strings, and the role of Buffers in Streams.
10. Explain Asynchronous File Operations in Node.js. Compare Promise-based (`fs/promises`) and Callback-based (`fs`) approaches with suitable examples. Discuss the advantages of using `async/await` and the problem of callback hell.

MODULE-V

Short Questions

1. What is MongoDB? Mention any two advantages of MongoDB.
2. What is MongoDB Atlas? Why is it used?
3. What is the use of the `.env` file in a Node.js application?
4. What is Mongoose? Mention one advantage of using Mongoose.
5. What is a Schema in Mongoose? Why is it used?
6. What is the difference between `find()` and `findById()` in Mongoose?
7. What is the difference between a Mongoose Schema and a Mongoose Model?
8. What is password hashing? Why should passwords not be stored in plain text?
9. What is JWT? Why is it used in authentication?
10. What is the difference between authentication and authorization?



LONG QUESTIONS:

1. Explain the steps to connect MongoDB Atlas with a Node.js Express application.
2. What is Mongoose? Explain any four advantages of using Mongoose with MongoDB.
3. Explain the MVC architecture and describe the responsibilities of the Model, View, and Controller with a suitable example.
4. Explain how to create a User Schema and Model using Mongoose.
5. Explain how to perform Create, Read, Update, and Delete operations on users using Mongoose.
6. Explain how to get all users and get a single user by ID using Mongoose.
7. Explain how to use query parameters in Mongoose to filter products based on price and category, and search products by name.
8. Explain the purpose of HTTP status codes 200, 201, 400, 401, 403, 404, and 500 in REST APIs
9. Explain the process of user registration and login using JWT authentication. Describe the major steps from registration to generating a JWT token after successful login.
10. Explain why passwords should not be stored as plain text. Describe how bcryptjs can be used to hash a password during registration and compare it during login.